Trust Gate — Complete Product Implementation To-Do List

Objective

Transform the existing `devsecops-dissertation` repository from a Python-focused security-scanning prototype into a production-ready, local-first application-security decision platform.

The finished product must:

- Run applicable security scanners reliably.
- Detect scanner failures instead of reporting false clean results.
- Normalise and deduplicate findings.
- Prioritise findings using transparent evidence.
- Support pull-request-native results.
- Gate only meaningful new risk by default.
- Generate standard security artefacts.
- Support multiple languages and scanner plugins.
- Provide safe remediation workflows.
- Maintain source-code privacy.
- Be commercially deployable.

Implementation rules

The implementing agent must follow these rules throughout the project.

- Complete phases in the listed order.
- Do not skip tests or acceptance criteria.
- Do not describe a task as complete until its validation checks pass.
- Use feature branches and pull requests for major phases.
- Preserve backward compatibility where reasonably possible.
- Do not silently ignore scanner, parser or report-generation failures.
- Do not classify missing reports as zero findings.
- Do not calculate high confidence from statistically insufficient samples.
- Do not overwrite raw scanner data during normalisation.
- Do not upload customer source code without explicit configuration.
- Pin all production dependencies and external actions.
- Document all architecture and security decisions.
- Update the changelog after each phase.
- Add automated tests for every significant behaviour.

Phase 0 — Repository audit and project preparation

0.1 Create a full repository inventory

- [ ] Inspect every repository file and directory.
- [ ] Record the purpose of every script, workflow and configuration file.
- [ ] Identify dead code, duplicated workflows and obsolete research artefacts.
- [ ] Identify proprietary and open-source components.
- [ ] Identify all external dependencies and GitHub Actions.
- [ ] Identify all current data flows.
- [ ] Identify all places where errors are ignored.
- [ ] Identify all current security assumptions.
- [ ] Identify all existing tests and test gaps.

Deliverables

- [ ] `docs/audits/REPOSITORY_AUDIT.md`
- [ ] `docs/audits/DEPENDENCY_INVENTORY.md`
- [ ] `docs/audits/DATA_FLOW_AUDIT.md`
- [ ] `docs/audits/TEST_GAP_ANALYSIS.md`

Completion criteria

- [ ] Every production file is accounted for.
- [ ] Every scanner dependency is documented.
- [ ] Every fail-open behaviour is documented.
- [ ] Every current product limitation is documented.

0.2 Establish the target repository structure

Create or migrate toward this structure:

```
trustgate/
├── action.yml
├── pyproject.toml
├── requirements/
│   ├── runtime.lock
│   ├── development.lock
│   └── scanners.lock
├── src/
│   └── trustgate/
│       ├── cli/
│       ├── adapters/
│       ├── aggregation/
│       ├── scoring/
│       ├── policies/
```

```
│       ├── findings/
│       ├── reporting/
│       ├── remediation/
│       ├── threat_intelligence/
│       ├── reachability/
│       ├── provenance/
│       └── licensing/
├── schemas/
├── policies/
├── benchmarks/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── fixtures/
│   ├── security/
│   └── end_to_end/
├── docs/
├── examples/
├── scripts/
└── .github/
    └── workflows/
```

- [ ] Move reusable production logic into `src/trustgate/`.
- [ ] Keep thin wrapper scripts only where required.
- [ ] Separate benchmark fixtures from production code.
- [ ] Separate licensing logic from security-scoring logic.
- [ ] Ensure package imports work from any working directory.
- [ ] Add a formal Python package configuration.
- [ ] Add a command-line entry point named `trustgate`.

Completion criteria

- [ ] `trustgate --help` runs successfully.
- [ ] Existing Action functionality still works.
- [ ] No production module relies on hard-coded working-directory paths.

0.3 Separate the research identity from the product identity

- [ ] Choose the final product name.
- [ ] Rename the public-facing Action.
- [ ] Create a new product-focused README.
- [ ] Move dissertation-specific material into `docs/research/`.
- [ ] Clearly distinguish experimental results from production claims.
- [ ] Remove unsupported phrases such as “works on any repository.”
- [ ] Describe the current release accurately as Python-first until broader support is finished.
- [ ] Define semantic versioning rules.
- [ ] Add a migration notice for existing users.

Deliverables

- [ ] `README.md`
- [ ] `docs/research/README.md`
- [ ] `docs/MIGRATION.md`
- [ ] `docs/VERSIONING.md`
- [ ] `CHANGELOG.md`

Completion criteria

- [ ] Product claims match actual functionality.
- [ ] Research findings cannot be confused with current production benchmarks.
- [ ] Existing users have documented migration steps.

Phase 1 — Build a secure and reliable foundation

1.1 Pin all dependencies

- [ ] Pin Python dependencies to exact versions.
- [ ] Generate hashes for Python package installations.
- [ ] Replace floating Docker image tags with immutable digests.
- [ ] Pin third-party GitHub Actions to commit SHAs.
- [ ] Record the human-readable version beside each pinned SHA.
- [ ] Add automated dependency-update tooling.
- [ ] Add a scanner compatibility matrix.
- [ ] Add a process for testing dependency upgrades before release.

Required files

- [ ] `requirements/runtime.lock`
- [ ] `requirements/development.lock`
- [ ] `requirements/scanners.lock`
- [ ] `docs/SCANNER_COMPATIBILITY.md`

Completion criteria

- [ ] No production dependency uses `latest`, `master` or an unbounded version.
- [ ] Every third-party Action is pinned to a commit SHA.
- [ ] A clean environment produces the same installed dependency versions.

1.2 Implement scanner-health states

Create these scanner execution states:

```
PASSED
FINDINGS_DETECTED
FAILED_SCANNER
```

```
TIMED_OUT
PARTIAL
SKIPPED_NOT_APPLICABLE
CANCELLED
```

- [ ] Create a common scanner execution result model.
- [ ] Record scanner start and end times.
- [ ] Record exit code.
- [ ] Record timeout status.
- [ ] Record whether the expected report was produced.
- [ ] Record parser status.
- [ ] Record scanner version.
- [ ] Record execution logs or log references.
- [ ] Stop treating missing reports as empty reports.
- [ ] Add configurable scanner timeouts.
- [ ] Add configurable scanner failure policy.

Example:

```
with:
  scanner-failure-policy: fail
  scanner-timeout-seconds: 900
```

Supported failure policies:

```
fail
warn
ignore
```

Default:

```
fail
```

Completion criteria

- [ ] A failed scanner cannot produce a “clean” gate.
- [ ] A missing report produces `FAILED_SCANNER`.
- [ ] A malformed report produces `FAILED_SCANNER` or `PARTIAL`.
- [ ] Required scanner failures block the build by default.
- [ ] Optional scanners can be configured separately.

1.3 Replace broad `|| true` handling

- [ ] Remove unconditional `|| true` from scanner execution.
- [ ] Capture scanner exit codes explicitly.
- [ ] Distinguish “findings found” from “scanner crashed.”

- [ ] Interpret each scanner's exit-code contract correctly.
- [ ] Add per-scanner execution wrappers.
- [ ] Store standard output and standard error separately where practical.
- [ ] Ensure scanner failures remain visible in GitHub Actions.

Completion criteria

- [ ] Every scanner wrapper has unit tests for success, findings, failure and timeout.
- [ ] No scanner crash is converted into a zero-finding result.

1.4 Harden workflow permissions

- [ ] Apply least-privilege GitHub workflow permissions.
- [ ] Use read-only permissions unless write access is required.
- [ ] Separate publishing workflows from scanning workflows.
- [ ] Prevent untrusted pull-request code from accessing secrets.
- [ ] Review all `pull_request_target` usage.
- [ ] Prevent unsafe interpolation of user-controlled inputs.
- [ ] Validate all Action inputs.
- [ ] Reject path traversal outside the workspace.
- [ ] Reject command-injection payloads.
- [ ] Sanitize artifact names.
- [ ] Validate URLs before DAST use.

Required documentation

- [ ] `docs/security/WORKFLOW_SECURITY.md`
- [ ] `docs/security/THREAT_MODEL.md`

Completion criteria

- [ ] Security tests cover malicious `target`, `fail-on`, URL and file-path inputs.
- [ ] Untrusted pull requests cannot access licence keys or publishing credentials.
- [ ] Workflow permissions pass an OpenSSF Scorecard review.

1.5 Add signed releases and provenance

- [ ] Produce versioned release archives.
- [ ] Generate release checksums.
- [ ] Sign release artefacts.
- [ ] Generate build provenance.
- [ ] Generate an SBOM for the product itself.
- [ ] Publish release attestations.
- [ ] Document verification commands for users.
- [ ] Add a release workflow requiring protected approval.

Completion criteria

- [ ] Users can verify that an artefact came from the official repository.

- [ ] Users can verify which commit and workflow produced the release.
- [ ] Every release contains an SBOM.

Phase 2 — Create a canonical finding model

2.1 Define the finding schema

Create a versioned JSON Schema for normalised findings.

Required fields:

```
schema_version
finding_id
fingerprint
scanner
scanner_version
rule_id
rule_version
category
cwe
cve
ghsa
osv
title
description
original_severity
normalised_severity
severity_reason
confidence
confidence_method
confidence_sample_size
confidence_interval
file
start_line
end_line
symbol
source
sink
data_flow
dependency
dependency_scope
reachability
environment
introduced_commit
first_seen
last_seen
status
evidence
```

```
remediation
raw_report_reference
```

- [ ] Create `schemas/finding.schema.json`.
- [ ] Create `schemas/scan-run.schema.json`.
- [ ] Create `schemas/policy-result.schema.json`.
- [ ] Create schema validation utilities.
- [ ] Version all schemas.
- [ ] Add backward-compatible migration functions.
- [ ] Validate all generated output before publishing.

Completion criteria

- [ ] Every finding produced by every adapter validates against the schema.
- [ ] Invalid findings are reported as parser errors.
- [ ] Schema migrations have tests.

2.2 Preserve original scanner data

- [ ] Preserve original severity.
- [ ] Preserve original description.
- [ ] Preserve original identifiers.
- [ ] Preserve the original scanner report.
- [ ] Record normalisation transformations separately.
- [ ] Never discard raw evidence needed for audit.
- [ ] Add an optional redaction layer for sensitive content.

Completion criteria

- [ ] Users can trace every normalised field to its original scanner output.
- [ ] Severity transformations are explainable.

2.3 Correct severity normalisation

- [ ] Stop assigning all dependency findings `HIGH`.
- [ ] Stop assigning all secrets `HIGH`.
- [ ] Read CVSS data where available.
- [ ] Preserve scanner-provided severity.
- [ ] Add source-specific severity mapping.
- [ ] Document every mapping rule.
- [ ] Add a confidence indicator for severity quality.
- [ ] Treat unknown severity as unknown, not low.
- [ ] Allow policy decisions based on original and normalised severity.

Completion criteria

- [ ] Each scanner has a tested severity mapping.
- [ ] Dependency severity reflects available advisory data.

- [ ] Secret severity considers type and validation status.

2.4 Implement stable finding fingerprints

Create fingerprints using relevant combinations of:

- Scanner-independent category
- CWE
- File path
- Symbol
- Source and sink
- Normalised code region
- Dependency name and version
- CVE or advisory ID
- Secret type
- Infrastructure resource identifier

- [ ] Add versioned fingerprint algorithms.

- [ ] Keep fingerprints stable across line-number changes.
- [ ] Avoid merging unrelated findings.
- [ ] Support fingerprint migration.
- [ ] Add collision tests.
- [ ] Add repository-relative path normalisation.

Completion criteria

- [ ] The same issue remains identifiable after nearby code changes.
- [ ] Duplicate reports from different scanners can be correlated.
- [ ] Fingerprint collisions are covered by tests.

Phase 3 — Repair the confidence and evidence methodology

3.1 Create one source of truth for benchmarks

- [ ] Remove manually duplicated metrics.
- [ ] Generate README metrics from benchmark artefacts.
- [ ] Generate documentation tables automatically.
- [ ] Version every benchmark dataset.
- [ ] Version scanner configurations.
- [ ] Version scanner rules.
- [ ] Store benchmark run metadata.
- [ ] Store the exact commit used for each result.
- [ ] Store the calculation method.

- [ ] Prevent publication when metrics are inconsistent.

Required structure

```
benchmarks/
├── datasets/
├── ground_truth/
├── configurations/
├── results/
├── reports/
└── manifests/
```

Completion criteria

- [ ] README, dashboard and research documents use the same generated metrics.
- [ ] Contradictory precision figures cannot be released.

3.2 Replace proximity-only ground-truth matching

- [ ] Stop relying solely on plus-or-minus-five-line matching.
- [ ] Match by vulnerability ID where fixtures support it.
- [ ] Match by file and symbol.
- [ ] Match by CWE.
- [ ] Match by source and sink.
- [ ] Match by code-region hash.
- [ ] Support manual adjudication.
- [ ] Record the matching reason.
- [ ] Record ambiguous matches.
- [ ] Require review before ambiguous results affect published metrics.

Completion criteria

- [ ] Dense files do not produce accidental true-positive matches.
- [ ] Every benchmark match is explainable.
- [ ] Ambiguous matches are excluded until adjudicated.

3.3 Implement statistically valid confidence

Use a Beta-Binomial or Wilson lower-bound approach.

Recommended initial approach:

```
displayed_estimate = posterior_mean
gating_estimate = 95% lower credible bound
```

Suggested prior:

```
Beta(1, 1)
```

The methodology must be configurable and documented.

- [ ] Implement posterior precision calculation.
- [ ] Implement confidence or credible intervals.
- [ ] Add sample-size maturity levels.
- [ ] Prevent small samples from receiving high-confidence classification.
- [ ] Add calibration-quality indicators.
- [ ] Track false negatives as well as false positives.
- [ ] Calculate precision, recall, F1, Brier score and calibration error.
- [ ] Include methodology version in every score.

Suggested maturity bands:

```
Experimental: n < 5
Directional: 5 ≤ n < 30
Moderate: 30 ≤ n < 100
Mature: n ≥ 100
Verified: independently reproduced
```

Completion criteria

- [ ] A rule with one true positive and zero false positives is not labelled “high confidence.”
- [ ] Every score includes sample size and interval.
- [ ] Gate decisions use the conservative bound.
- [ ] Statistical tests cover edge cases.

3.4 Separate confidence concepts

Do not use one field for all confidence types.

Create separate fields for:

```
scanner_rule_reliability
finding_validity_confidence
reachability_confidence
exploitability_confidence
remediation_confidence
overall_decision_confidence
```

- [ ] Define each confidence type.
- [ ] Define which evidence affects each type.
- [ ] Prevent circular calculations.
- [ ] Display each component in reports.
- [ ] Do not hide them behind one unexplained score.

Completion criteria

- [ ] Users can see why a decision was made.
- [ ] Scanner reliability is not presented as exploitability probability.

Phase 4 — Build the scanner adapter system

4.1 Define the adapter interface

Each adapter must implement:

```
class ScannerAdapter:
    def metadata(self): ...
    def is_applicable(self, repository_context): ...
    def prepare(self, context): ...
    def execute(self, target, context): ...
    def health_check(self, execution): ...
    def parse(self, report): ...
    def normalize(self, finding): ...
    def fingerprint(self, finding): ...
    def cleanup(self, context): ...
```

Adapter metadata must include:

```
name
version
category
supported_languages
supported_files
required_runtime
default_timeout
licence
data_leaves_runner
report_format
capabilities
```

- [ ] Implement a base adapter class.
- [ ] Add typed interfaces.
- [ ] Add adapter registration.
- [ ] Add adapter discovery.
- [ ] Add adapter configuration.
- [ ] Add adapter-level tests.
- [ ] Add adapter failure isolation.
- [ ] Add an adapter SDK guide.

Required documentation

- [ ] `docs/ADAPTER_SDK.md`

Completion criteria

- [ ] A new scanner can be added without modifying the aggregator core.
- [ ] Broken adapters do not corrupt other results.

4.2 Migrate existing scanners into adapters

Create adapters for:

- [ ] Bandit
- [ ] Semgrep
- [ ] pip-audit
- [ ] Trivy
- [ ] Gitleaks
- [ ] OWASP ZAP

Each adapter must include:

- [ ] Applicability detection
- [ ] Version detection
- [ ] Execution wrapper
- [ ] Timeout handling
- [ ] Health validation
- [ ] Parser
- [ ] Severity mapping
- [ ] Fingerprinting
- [ ] Test fixtures
- [ ] Error fixtures
- [ ] Malformed report fixtures

Completion criteria

- [ ] All existing functionality uses adapters.
- [ ] No scanner-specific parsing remains inside the central aggregator.

4.3 Add additional scanner integrations

Add adapters in this order:

- [ ] OSV-Scanner
- [ ] Syft
- [ ] Grype
- [ ] Checkov or KICS
- [ ] Hadolint
- [ ] Gosec

- [ ] Brakeman
- [ ] SpotBugs or equivalent Java support
- [ ] JavaScript and TypeScript security scanning
- [ ] TruffleHog as an optional secret-validation source
- [ ] CodeQL SARIF import

Completion criteria

- [ ] Each integration has documented applicability.
- [ ] Unsupported repositories do not run irrelevant scanners.
- [ ] New adapters pass the common adapter test suite.

Phase 5 — Add intelligent repository detection

5.1 Build repository context detection

Detect:

- [ ] Languages
- [ ] Frameworks
- [ ] Package managers
- [ ] Lock files
- [ ] Build systems
- [ ] Container files
- [ ] Kubernetes files
- [ ] Terraform files
- [ ] CloudFormation files
- [ ] OpenAPI specifications
- [ ] Test directories
- [ ] Generated files
- [ ] Vendored dependencies
- [ ] Monorepo packages
- [ ] Runtime and development dependencies

Completion criteria

- [ ] The scan plan accurately explains why each scanner was selected.
- [ ] Generated and vendored files can be excluded safely.
- [ ] Monorepos generate per-package scan contexts.

5.2 Generate an explicit scan plan

Before executing scanners, output:

```
Detected technologies
Enabled scanners
Skipped scanners
```

```
Reason for each decision
Target directories
Expected outputs
Timeouts
Data-handling behaviour
```

- [ ] Add `trustgate plan`.
- [ ] Add `--dry-run`.
- [ ] Add JSON and human-readable output.
- [ ] Allow users to override automatic detection.
- [ ] Validate conflicting configuration.

Completion criteria

- [ ] Users can inspect the complete scan plan before execution.
- [ ] Automatic decisions are transparent.

Phase 6 — Deduplicate and correlate findings

6.1 Implement exact deduplication

- [ ] Merge repeated findings from the same scanner.
- [ ] Handle repeated locations.
- [ ] Preserve occurrence counts.
- [ ] Preserve all raw evidence references.

6.2 Implement cross-scanner correlation

Correlate findings by:

- [ ] CWE
- [ ] File
- [ ] Symbol
- [ ] Source
- [ ] Sink
- [ ] Code region
- [ ] Dependency
- [ ] CVE
- [ ] Infrastructure resource
- [ ] Secret fingerprint

Create scanner-agreement fields:

```
supporting_scanners
contradicting_scanners
```

```
agreement_strength
correlation_reason
```

Completion criteria

- [ ] Bandit and Semgrep reports of the same SQL injection become one consolidated issue.
- [ ] Supporting evidence from both scanners remains available.
- [ ] Unrelated findings are not incorrectly merged.

6.3 Add evidence-weighted corroboration

- [ ] Increase finding-validity confidence when independent scanners agree.
- [ ] Avoid double-counting scanners using the same rule source.
- [ ] Track shared rule ancestry where known.
- [ ] Record DAST confirmation separately.
- [ ] Record human confirmation separately.
- [ ] Add confidence limits to corroboration calculations.

Completion criteria

- [ ] Corroboration logic is documented and tested.
- [ ] Scanner agreement does not automatically imply exploitability.

Phase 7 — Add threat-intelligence enrichment

7.1 Add advisory enrichment

Integrate:

- [ ] OSV
- [ ] GitHub Security Advisories
- [ ] NVD where needed
- [ ] EPSS
- [ ] CISA KEV

Store:

```
advisory IDs
CVSS score
CVSS vector
EPSS probability
EPSS percentile
KEV status
known exploitation date
ransomware association
fixed versions
```

```
published date
modified date
data-source timestamp
```

Completion criteria

- [ ] Enrichment failures are visible.
- [ ] Cached data has an expiry policy.
- [ ] Gate results identify stale threat data.
- [ ] No threat feed is treated as complete risk context.

7.2 Add offline and privacy-preserving modes

- [ ] Support local threat-data cache.
- [ ] Support fully offline runs.
- [ ] Document which identifiers are sent externally.
- [ ] Add `network-mode: disabled`.
- [ ] Add `network-mode: metadata-only`.
- [ ] Add `network-mode: full`.
- [ ] Default to the least invasive mode appropriate for the feature.

Completion criteria

- [ ] Source code is never sent to enrichment services.
- [ ] Offline scans remain fully functional with cached data.

Phase 8 — Add reachability analysis

8.1 Implement dependency reachability

Start with Python, then expand.

Determine:

- [ ] Whether a vulnerable package is installed.
- [ ] Whether it is a direct or transitive dependency.
- [ ] Whether it is imported.
- [ ] Whether the vulnerable symbol is called.
- [ ] Whether it is production or development-only.
- [ ] Whether it is included in the deployed artefact.
- [ ] Whether a call path exists.
- [ ] Whether analysis is incomplete.

Use statuses:

```
CONFIRMED_REACHABLE
LIKELY_REACHABLE
NO_PATH_FOUND
NOT_ANALYSED
ANALYSIS_INCOMPLETE
DYNAMIC_BEHAVIOUR_UNKNOWN
```

Completion criteria

- [ ] “No path found” is never described as “not exploitable.”
- [ ] Reachability evidence includes the analysed call path.
- [ ] Dynamic limitations are visible.

8.2 Implement SAST source-to-sink analysis

For supported languages:

- [ ] Identify untrusted sources.
- [ ] Identify sanitizers.
- [ ] Identify dangerous sinks.
- [ ] Trace intra-file data flow.
- [ ] Trace cross-file data flow.
- [ ] Trace framework routing.
- [ ] Record authentication requirements.
- [ ] Record authorization checks where detectable.
- [ ] Record path confidence.
- [ ] Show source-to-sink evidence.

Completion criteria

- [ ] Supported findings can show an explainable data-flow trace.
- [ ] Unsupported analysis is marked explicitly.

8.3 Correlate static and dynamic evidence

- [ ] Match DAST endpoints to source-code routes.
- [ ] Match DAST parameters to SAST sources.
- [ ] Match DAST proof to SAST sinks.
- [ ] Increase priority when a static issue is dynamically confirmed.
- [ ] Record failed reproduction attempts without marking the issue false.
- [ ] Distinguish blocked authentication from failed exploitation.

Completion criteria

- [ ] Dynamically confirmed findings show both static and runtime evidence.
- [ ] Inconclusive DAST results do not automatically suppress SAST findings.

Phase 9 — Package DAST safely

9.1 Add reusable DAST configuration

Support:

```
with:
  dast-enabled: true
  dast-target-url: ${{ steps.deploy.outputs.preview-url }}
  dast-mode: baseline
  dast-openapi-path: openapi.yaml
  dast-auth-type: bearer
  dast-auth-secret: ${{ secrets.DAST_TOKEN }}
```

- [ ] Support baseline mode.
- [ ] Support API mode.
- [ ] Support authenticated mode.
- [ ] Support preview environments.
- [ ] Support scope allowlists.
- [ ] Reject non-allowlisted domains.
- [ ] Add rate limits.
- [ ] Add request limits.
- [ ] Add maximum scan duration.
- [ ] Add safe and active scan modes.
- [ ] Require explicit opt-in for active scans.
- [ ] Prevent scanning production by accident.
- [ ] Require acknowledgement for public targets.

Completion criteria

- [ ] DAST cannot target arbitrary external domains by default.
- [ ] Active scanning requires explicit configuration.
- [ ] Authentication secrets are redacted from logs.

Phase 10 — Build contextual decision scoring

10.1 Define decision components

The engine must consider:

- [ ] Finding-validity confidence
- [ ] Original severity
- [ ] Normalised severity
- [ ] Reachability
- [ ] EPSS
- [ ] CISA KEV

- [ ] Public exploit availability
- [ ] Internet exposure
- [ ] Authentication requirements
- [ ] Data sensitivity
- [ ] Asset criticality
- [ ] Runtime environment
- [ ] Existing controls
- [ ] Fix availability
- [ ] New versus existing status
- [ ] Human triage state

Do not collapse these into an unexplained number.

10.2 Create decision outcomes

Support:

```
BLOCK_IMMEDIATELY
FIX_BEFORE_RELEASE
FIX_WITHIN_SLA
INVESTIGATE
MONITOR
TEMPORARILY_SUPPRESSED
ACCEPTED_RISK
LIKELY_NOISE
INSUFFICIENT_EVIDENCE
```

- [ ] Document rules for each outcome.
- [ ] Make outcomes policy-driven.
- [ ] Show the complete explanation.
- [ ] Include evidence strength.
- [ ] Include unresolved uncertainty.
- [ ] Include policy version.

Example explanation:

```
Blocked because:
- Confirmed reachable SQL injection
- Unauthenticated internet-facing endpoint
- Detected by Bandit and Semgrep
- DAST reproduced the issue
- Confidence lower bound: 0.91
- Policy: production-injection-v2
```

Completion criteria

- [ ] Every decision is reproducible from stored evidence.
- [ ] Users can inspect which policy caused the result.

Phase 11 — Implement policy-as-code

11.1 Define the policy schema

Policies must support:

- [ ] Severity
- [ ] CWE
- [ ] CVE
- [ ] EPSS
- [ ] KEV
- [ ] Reachability
- [ ] Environment
- [ ] Repository
- [ ] Branch
- [ ] Asset criticality
- [ ] Confidence lower bound
- [ ] Finding status
- [ ] Introduced-in-PR status
- [ ] Fix availability
- [ ] Scanner health
- [ ] Secret validation status
- [ ] Suppression expiry

Example

```
version: 1

policies:
  - name: block-exploitable-production-risk
    action: block
    when:
      any:
        - all:
            - environment: production
            - kev: true

        - all:
            - severity: critical
            - reachability: confirmed

        - all:
            - cwe:
                - CWE-78
                - CWE-89
                - CWE-94
```

```
      - confidence_lower_bound: ">=0.80"
      - introduced_in_pull_request: true
```

11.2 Add policy tooling

- [ ] Add policy validation.
- [ ] Add policy simulation.
- [ ] Add policy explanation.
- [ ] Add policy unit testing.
- [ ] Add policy versioning.
- [ ] Add policy inheritance.
- [ ] Add repository overrides.
- [ ] Add organisation defaults.
- [ ] Prevent invalid rules from silently passing.

Commands:

```
trustgate policy validate
trustgate policy test
trustgate policy explain
trustgate policy simulate
```

Completion criteria

- [ ] A policy can be tested against saved findings before deployment.
- [ ] Invalid policies fail clearly.
- [ ] Policy decisions are deterministic.

11.3 Create standard policy packs

- [ ] Startup baseline
- [ ] High-assurance baseline
- [ ] Financial services
- [ ] Healthcare
- [ ] Public-sector supplier
- [ ] OWASP ASVS-aligned
- [ ] NIST SSDF-aligned
- [ ] Container security
- [ ] Secret protection
- [ ] Supply-chain security

Completion criteria

- [ ] Every policy pack has documentation and tests.
- [ ] Policy packs state that automated evidence does not guarantee compliance.

Phase 12 — Add baseline and differential scanning

12.1 Create baseline support

- [ ] Generate a baseline from the default branch.
- [ ] Store baseline findings by fingerprint.
- [ ] Compare pull-request findings to the baseline.
- [ ] Detect new findings.
- [ ] Detect removed findings.
- [ ] Detect worsened findings.
- [ ] Detect newly reachable findings.
- [ ] Detect newly exploited dependencies.
- [ ] Detect expired suppressions.
- [ ] Detect scanner coverage regressions.

12.2 Gate new risk by default

Default behaviour:

```
Block newly introduced qualifying risk.
Report legacy risk without immediately blocking adoption.
```

- [ ] Add `gate-mode: new`.
- [ ] Add `gate-mode: all`.
- [ ] Add `gate-mode: worsened`.
- [ ] Add `gate-mode: policy`.
- [ ] Allow explicit legacy-risk enforcement.
- [ ] Show baseline age.
- [ ] Fail when the baseline is invalid or incompatible.

Completion criteria

- [ ] Existing repositories can adopt the product without immediately fixing every historical finding.
- [ ] Newly introduced high-risk findings still block the pull request.

Phase 13 — Build the finding lifecycle

13.1 Implement finding states

Support:

```
OPEN
CONFIRMED
INVESTIGATING
FIX_IN_PROGRESS
FIXED
MITIGATED
RISK_ACCEPTED
FALSE_POSITIVE
TEMPORARILY_SUPPRESSED
REOPENED
```

- [ ] Record state history.
- [ ] Record actor.
- [ ] Record timestamp.
- [ ] Record reason.
- [ ] Record evidence.
- [ ] Record approval where required.
- [ ] Record expiry.
- [ ] Support automatic reopening.

13.2 Implement suppressions

Every suppression must require:

- [ ] Finding fingerprint
- [ ] Reason
- [ ] Author
- [ ] Creation date
- [ ] Expiry date
- [ ] Scope
- [ ] Approval
- [ ] Evidence
- [ ] Revalidation rule
- [ ] Prevent permanent suppression by default.
- [ ] Add suppression linting.
- [ ] Add suppression-expiry warnings.
- [ ] Reopen when code meaningfully changes.
- [ ] Reopen when reachability changes.
- [ ] Reopen when KEV status changes.
- [ ] Reopen when exploit evidence changes.
- [ ] Reopen when policy changes.

Completion criteria

- [ ] Expired suppressions automatically re-enter evaluation.
- [ ] Every suppression is auditable.

- [ ] A suppression cannot silently apply to unrelated findings.

Phase 14 — Add GitHub-native integration

14.1 Generate SARIF

- [ ] Map supported findings to SARIF 2.1.0.
- [ ] Validate generated SARIF.
- [ ] Include rule metadata.
- [ ] Include precise locations.
- [ ] Include severity.
- [ ] Include remediation guidance.
- [ ] Include fingerprints.
- [ ] Include partial fingerprints.
- [ ] Upload results to GitHub code scanning.

Completion criteria

- [ ] Findings appear in GitHub's Security tab.
- [ ] Findings annotate pull-request code where locations exist.

14.2 Add GitHub Checks integration

Create one consolidated check containing:

- [ ] Gate result
- [ ] Scanner-health summary
- [ ] New findings
- [ ] Blocking findings
- [ ] Suppressed findings
- [ ] Unscored findings
- [ ] Evidence explanations
- [ ] Links to detailed artefacts
- [ ] Policy information
- [ ] Baseline comparison

Completion criteria

- [ ] Developers can understand the release decision without downloading an artefact.
- [ ] Branch protection can require the Trust Gate check.

14.3 Add pull-request comments carefully

- [ ] Post one consolidated comment.
- [ ] Update the existing comment instead of creating duplicates.
- [ ] Keep the summary concise.

- [ ] Collapse long details.
- [ ] Link to exact code locations.
- [ ] Avoid exposing secrets.
- [ ] Avoid posting proprietary source excerpts unnecessarily.
- [ ] Include remediation status.

Completion criteria

- [ ] Repeated runs update one comment.
- [ ] Pull requests are not flooded with scanner messages.

Phase 15 — Generate standard security artefacts

15.1 Generate SBOMs

Support:

- [ ] CycloneDX JSON
- [ ] SPDX JSON

Include:

- [ ] Direct dependencies
- [ ] Transitive dependencies
- [ ] Versions
- [ ] Licences
- [ ] Package URLs
- [ ] Hashes
- [ ] Dependency relationships

15.2 Generate VEX

- [ ] Generate CycloneDX VEX.
- [ ] Record exploitability status.
- [ ] Record justification.
- [ ] Record analysis state.
- [ ] Link VEX decisions to reachability evidence.
- [ ] Link VEX decisions to approvals.
- [ ] Version and sign VEX output.

15.3 Generate compliance and audit evidence

Produce a signed evidence bundle containing:

- [ ] Commit SHA
- [ ] Repository

- [ ] Workflow identity
- [ ] Timestamp
- [ ] Scanner versions
- [ ] Scanner health
- [ ] Scan configuration
- [ ] Policy version
- [ ] Baseline version
- [ ] Findings
- [ ] Suppressions
- [ ] Approvals
- [ ] Gate result
- [ ] SBOM
- [ ] VEX
- [ ] Provenance
- [ ] Attestation
- [ ] Exclusions
- [ ] Data-source timestamps

Completion criteria

- [ ] Audit evidence is reproducible and verifiable.
- [ ] Reports distinguish automated evidence from manual compliance requirements.

Phase 16 — Add safe remediation

16.1 Implement deterministic remediation

Begin with predictable fixes:

- [ ] Parameterised SQL queries
- [ ] Removal of `shell=True`
- [ ] Safe YAML loading
- [ ] Replacement of weak hashing
- [ ] Dependency upgrades
- [ ] Secure Docker user configuration
- [ ] Removal of exposed secrets
- [ ] Secure HTTP-header configuration

Each fixer must include:

- [ ] Supported rule IDs
- [ ] Supported frameworks
- [ ] Preconditions
- [ ] Transformation
- [ ] Tests
- [ ] Rollback behaviour
- [ ] Risk notes

Completion criteria

- [ ] Deterministic fixes are covered by before-and-after tests.
- [ ] Unsupported code is not modified.

16.2 Implement guided remediation

For every supported finding, provide:

- [ ] Why it is vulnerable
- [ ] Exploit scenario
- [ ] Relevant source and sink
- [ ] Secure coding pattern
- [ ] Framework-specific example
- [ ] CWE reference
- [ ] Testing guidance
- [ ] Regression risks
- [ ] Verification instructions

16.3 Add AI-assisted remediation

AI remediation must be optional.

- [ ] Require explicit opt-in.
- [ ] Display which code context leaves the runner.
- [ ] Support local-model mode.
- [ ] Support redaction.
- [ ] Generate patches on isolated branches.
- [ ] Run formatting.
- [ ] Run type checking.
- [ ] Run unit tests.
- [ ] Run integration tests.
- [ ] Rerun relevant security scanners.
- [ ] Verify the original finding disappeared.
- [ ] Check for new high-risk findings.
- [ ] Open a draft pull request.
- [ ] Mark the fix as unverified until all checks pass.

Completion criteria

- [ ] The product never claims an issue is fixed solely because AI generated a patch.
- [ ] Failed verification prevents automatic fix completion.

Phase 17 — Expand the benchmark corpus

17.1 Build a multilingual benchmark

Include:

- [ ] Python
- [ ] JavaScript
- [ ] TypeScript
- [ ] Java
- [ ] Go
- [ ] Ruby
- [ ] C#
- [ ] Infrastructure as Code
- [ ] Containers
- [ ] Kubernetes

For each language, include:

- [ ] Multiple frameworks
- [ ] True vulnerabilities
- [ ] Patched equivalents
- [ ] Safe lookalikes
- [ ] Cross-file cases
- [ ] Sanitised cases
- [ ] Reachable cases
- [ ] Unreachable cases
- [ ] Test-only code
- [ ] Development-only dependencies
- [ ] Production dependencies

17.2 Create robust labelling procedures

- [ ] Use two independent reviewers.
- [ ] Record reviewer decisions.
- [ ] Add adjudication for disagreements.
- [ ] Measure inter-rater agreement.
- [ ] Document labelling rules.
- [ ] Record uncertainty.
- [ ] Separate public and private benchmark partitions.
- [ ] Create blind evaluation sets.
- [ ] Prevent benchmark leakage into rule tuning.

Completion criteria

- [ ] Published benchmark claims are reproducible.
- [ ] Every labelled item has review evidence.
- [ ] Private evaluation sets remain separated from development data.

17.3 Automate benchmark execution

- [ ] Run benchmarks on supported scanner upgrades.
- [ ] Compare new and previous versions.
- [ ] Detect precision regressions.
- [ ] Detect recall regressions.
- [ ] Detect runtime regressions.
- [ ] Block releases when thresholds fail.
- [ ] Generate benchmark reports automatically.

Completion criteria

- [ ] Scanner upgrades cannot silently reduce detection quality.
- [ ] Published metrics always identify scanner and dataset versions.

Phase 18 — Add customer-specific calibration

18.1 Capture customer feedback

Support:

- [ ] Confirmed true positive
- [ ] Confirmed false positive
- [ ] Accepted risk
- [ ] Fixed
- [ ] Reopened
- [ ] Remediation accepted
- [ ] Remediation rejected
- [ ] Keep feedback local by default.
- [ ] Allow encrypted export.
- [ ] Allow deletion.
- [ ] Allow repository-specific calibration.
- [ ] Allow organisation-specific calibration.
- [ ] Keep global and customer-specific confidence separate.

18.2 Build local calibration models

- [ ] Calculate repository-specific rule reliability.
- [ ] Calculate organisation-specific rule reliability.
- [ ] Apply Bayesian shrinkage to avoid overfitting.
- [ ] Show global and local estimates together.
- [ ] Require minimum evidence before local overrides affect gating.
- [ ] Record model version.
- [ ] Add drift detection.

Completion criteria

- [ ] Small local samples do not create extreme confidence.
- [ ] Customers can inspect and reset calibration data.

Phase 19 — Build deployment modes

19.1 Local-only mode

- [ ] All scanning occurs in CI.
- [ ] Findings remain local.
- [ ] Policies remain local.
- [ ] Reporting remains local.
- [ ] Threat feeds are cached locally.
- [ ] No telemetry is sent without consent.

19.2 Hybrid mode

- [ ] Source code stays local.
- [ ] Only approved finding metadata is uploaded.
- [ ] Support field-level redaction.
- [ ] Support customer-managed encryption keys.
- [ ] Document exact transmitted fields.
- [ ] Add upload allowlists.

19.3 Self-hosted enterprise mode

- [ ] Containerised deployment.
- [ ] Database migration tooling.
- [ ] Backup and restore.
- [ ] SSO or SAML.
- [ ] SCIM.
- [ ] Role-based access control.
- [ ] Audit logging.
- [ ] Data-retention settings.
- [ ] Offline threat-data import.
- [ ] High-availability guidance.
- [ ] Security-hardening guide.

Completion criteria

- [ ] Every feature documents its data-handling behaviour.
- [ ] Enterprise users can operate without sending data to the vendor.

Phase 20 — Build team and organisation features

20.1 Create the management plane

- [ ] Multi-repository dashboard
- [ ] Organisation risk overview
- [ ] Repository trends
- [ ] Scanner health
- [ ] Policy compliance
- [ ] Mean time to remediation
- [ ] Finding ownership
- [ ] Suppression expiry
- [ ] Benchmark drift
- [ ] Threat-intelligence changes

20.2 Add integrations

- [ ] Linear
- [ ] Jira
- [ ] Slack
- [ ] Microsoft Teams
- [ ] Email
- [ ] Webhooks
- [ ] SIEM export
- [ ] Ticket synchronization

Completion criteria

- [ ] Findings can be assigned and tracked without duplicate tickets.
- [ ] Closing a validated ticket updates finding state safely.

Phase 21 — Add compliance mappings

Map evidence to:

- [ ] OWASP Top 10
- [ ] OWASP ASVS
- [ ] OWASP SAMM
- [ ] NIST SSDF
- [ ] CWE
- [ ] PCI DSS
- [ ] ISO 27001
- [ ] SOC 2
- [ ] Cyber Essentials

For every mapping:

- [ ] State what automated evidence supports.
- [ ] State what still requires manual verification.
- [ ] Do not claim complete compliance.
- [ ] Record mapping version.
- [ ] Add exportable evidence reports.

Completion criteria

- [ ] Reports say “evidence available” rather than automatically declaring compliance.
- [ ] Framework mappings are reviewed and versioned.

Phase 22 — Documentation and developer experience

22.1 Create complete user documentation

- [ ] Five-minute quick start
- [ ] GitHub Action installation
- [ ] CLI installation
- [ ] Configuration reference
- [ ] Policy reference
- [ ] Scanner compatibility
- [ ] DAST safety guide
- [ ] Offline operation
- [ ] Baseline setup
- [ ] Suppression workflow
- [ ] Remediation workflow
- [ ] Troubleshooting
- [ ] Security model
- [ ] Privacy model
- [ ] Upgrade guide
- [ ] Migration guide

22.2 Add working examples

Create examples for:

- [ ] Python Flask
- [ ] Python Django
- [ ] Node.js
- [ ] TypeScript
- [ ] Java
- [ ] Go
- [ ] Docker

- [ ] Terraform
- [ ] Kubernetes
- [ ] Monorepo
- [ ] Authenticated DAST
- [ ] Offline mode
- [ ] Custom policy
- [ ] Self-hosted deployment

Completion criteria

- [ ] Every documented example runs in CI.
- [ ] Broken examples block releases.

22.3 Improve error messages

Every error must include:

- [ ] What failed
- [ ] Why it likely failed
- [ ] Whether security coverage is incomplete
- [ ] Whether the gate is trustworthy
- [ ] How to resolve it
- [ ] Where logs are stored

Completion criteria

- [ ] Users do not need to inspect source code to understand common failures.

Phase 23 — Testing strategy

23.1 Unit tests

Cover:

- [ ] All parsers
- [ ] Severity normalisation
- [ ] Fingerprinting
- [ ] Deduplication
- [ ] Correlation
- [ ] Confidence calculations
- [ ] Policy evaluation
- [ ] Suppression expiry
- [ ] Baseline comparison
- [ ] Schema validation
- [ ] Threat enrichment
- [ ] Reachability states
- [ ] Remediation rules

Target:

```
At least 90% coverage for core decision and policy modules.
```

23.2 Integration tests

Test:

- [ ] Every scanner adapter
- [ ] Missing report
- [ ] Malformed report
- [ ] Scanner timeout
- [ ] Scanner crash
- [ ] Empty repository
- [ ] Unsupported repository
- [ ] Monorepo
- [ ] Offline mode
- [ ] Baseline comparison
- [ ] SARIF upload generation
- [ ] SBOM and VEX generation

23.3 Security tests

Test:

- [ ] Command injection
- [ ] Path traversal
- [ ] Malicious filenames
- [ ] Malicious scanner output
- [ ] HTML injection
- [ ] SARIF injection
- [ ] Secret leakage
- [ ] Licence bypass
- [ ] Policy bypass
- [ ] Suppression bypass
- [ ] Untrusted pull requests
- [ ] Symlink attacks
- [ ] Zip-slip attacks
- [ ] Resource exhaustion
- [ ] DAST target abuse
- [ ] Dependency confusion

23.4 End-to-end tests

Test complete flows:

- [ ] Clean repository passes.
- [ ] New critical vulnerability blocks.
- [ ] Legacy vulnerability reports but does not block in new-risk mode.
- [ ] Scanner failure blocks.
- [ ] DAST confirms a SAST issue.
- [ ] Suppression expires and reopens.
- [ ] KEV status changes and reprioritises.
- [ ] Deterministic fix passes verification.
- [ ] Failed fix remains unresolved.
- [ ] SARIF appears correctly.
- [ ] Evidence bundle verifies successfully.

Completion criteria

- [ ] End-to-end tests run on every release.
- [ ] A release cannot proceed when critical workflows fail.

Phase 24 — Performance and reliability

24.1 Improve execution performance

- [ ] Run independent scanners in parallel.
- [ ] Cache scanner installations safely.
- [ ] Cache threat data.
- [ ] Cache dependency graphs.
- [ ] Support changed-file scanning.
- [ ] Support incremental rescanning.
- [ ] Avoid rescanning unchanged packages.
- [ ] Add resource limits.
- [ ] Track scanner duration.

24.2 Define reliability targets

Initial service targets:

```
Core engine success rate: ≥99.5%
False-clean result rate from scanner failure: 0%
Median PR scan time: <5 minutes for supported small repositories
p95 PR scan time: <15 minutes
Policy evaluation time: <5 seconds after findings are available
```

- [ ] Add performance benchmarks.

- [ ] Add regression thresholds.
- [ ] Add reliability dashboards.
- [ ] Add failure-rate reporting.

Phase 25 — Product packaging and licensing

25.1 Define editions

Community

- [ ] Core scanners
- [ ] SARIF
- [ ] Basic security gate
- [ ] Local reports
- [ ] Standard policy packs

Pro

- [ ] Evidence-based prioritisation
- [ ] EPSS and KEV enrichment
- [ ] Differential scanning
- [ ] Deduplication
- [ ] Expiring suppressions
- [ ] Guided remediation
- [ ] Private repositories

Team

- [ ] Multi-repository dashboard
- [ ] Central policies
- [ ] Assignment
- [ ] Integrations
- [ ] Finding lifecycle
- [ ] Organisation calibration
- [ ] Audit logs

Enterprise

- [ ] Self-hosting
- [ ] SSO
- [ ] SCIM
- [ ] RBAC
- [ ] Data residency
- [ ] Compliance evidence
- [ ] Custom policy packs
- [ ] Enterprise SLA

25.2 Review licensing architecture

- [ ] Document which components are open source.
- [ ] Document which components are proprietary.
- [ ] Ensure proprietary functionality is not protected only through client-side orchestration.
- [ ] Ensure commercial terms are legally clear.
- [ ] Add licence compatibility review for every bundled scanner.
- [ ] Add a third-party notices file.
- [ ] Add subscription lifecycle tests.
- [ ] Add graceful offline validation.
- [ ] Add key rotation.
- [ ] Add revocation strategy where required.
- [ ] Do not allow licensing failure to corrupt security results.

Completion criteria

- [ ] Users can always access their raw security findings.
- [ ] Paid feature failure cannot produce an incorrect clean result.
- [ ] Third-party licences are documented.

Phase 26 — Security review and external validation

26.1 Perform internal security review

- [ ] Review threat model.
- [ ] Review cryptographic usage.
- [ ] Review licence verification.
- [ ] Review workflow security.
- [ ] Review parser trust boundaries.
- [ ] Review report rendering.
- [ ] Review external network calls.
- [ ] Review secret handling.
- [ ] Review update mechanisms.
- [ ] Review self-hosted deployment.

26.2 Arrange independent review

- [ ] External code audit
- [ ] Penetration test
- [ ] Statistical methodology review
- [ ] Benchmark labelling review
- [ ] Privacy review

- [ ] Licensing review

- [ ] Publish a responsible summary of findings.

- [ ] Fix all critical and high issues.
- [ ] Track medium issues with deadlines.
- [ ] Retest completed fixes.

Completion criteria

- [ ] No unresolved critical security findings.
- [ ] No unresolved high findings without documented executive acceptance.
- [ ] Statistical claims have independent review.

Phase 27 — Release readiness

27.1 Prepare release documentation

- [ ] Release notes
- [ ] Installation guide
- [ ] Migration guide
- [ ] Upgrade guide
- [ ] Compatibility matrix
- [ ] Known limitations
- [ ] Security policy
- [ ] Support policy
- [ ] Data-processing documentation
- [ ] Service-level commitments
- [ ] Incident-response process
- [ ] Responsible-disclosure process

27.2 Complete release verification

- [ ] All unit tests pass.
- [ ] All integration tests pass.
- [ ] All security tests pass.
- [ ] All end-to-end tests pass.
- [ ] Performance targets pass.
- [ ] Benchmark thresholds pass.
- [ ] Documentation examples pass.
- [ ] SBOM is generated.
- [ ] Provenance is generated.
- [ ] Artefacts are signed.
- [ ] Changelog is updated.
- [ ] Version is tagged.
- [ ] Release is reproducible.

Product metrics to implement

Track these metrics from the first production release:

- [ ] Precision by scanner
- [ ] Precision by rule
- [ ] Precision by language
- [ ] Precision by framework
- [ ] Recall by vulnerability category
- [ ] F1 score
- [ ] Confidence calibration error
- [ ] Brier score
- [ ] False-positive reduction
- [ ] False-negative escape rate
- [ ] Scanner failure rate
- [ ] Partial-scan rate
- [ ] Median scan duration
- [ ] p95 scan duration
- [ ] Time to first result
- [ ] Mean time to remediation
- [ ] Gate override rate
- [ ] Suppression expiry rate
- [ ] Reopened finding rate
- [ ] Autofix acceptance rate
- [ ] Autofix verification rate
- [ ] Developer hours saved
- [ ] Security engineer hours saved

Required final architecture outcome

The final data flow should resemble:

```
Repository checkout
        ↓
Repository context detection
        ↓
Transparent scan plan
        ↓
Applicable scanner adapters
        ↓
Scanner-health validation
        ↓
Raw report preservation
        ↓
Schema validation
        ↓
Finding normalisation
```

```
        ↓
Stable fingerprinting
        ↓
Deduplication and correlation
        ↓
Threat-intelligence enrichment
        ↓
Reachability analysis
        ↓
Evidence and confidence calculation
        ↓
Policy-as-code evaluation
        ↓
Release decision
        ↓
SARIF + Checks + PR summary
        ↓
SBOM + VEX + signed evidence bundle
        ↓
Optional verified remediation
```

Definition of fully complete

The project is fully complete only when all conditions below are true.

Security reliability

- [ ] Scanner failures cannot appear as clean scans.
- [ ] Missing or malformed reports are detected.
- [ ] Dependencies and Actions are pinned.
- [ ] Releases are signed and attested.
- [ ] The product has passed an independent security review.

Evidence quality

- [ ] Confidence scores use statistically defensible methods.
- [ ] Sample sizes and intervals are visible.
- [ ] Scanner reliability is separate from exploitability.
- [ ] Published benchmark figures are consistent.
- [ ] Benchmark results are reproducible.

Developer experience

- [ ] Installation is documented and tested.
- [ ] Results appear directly in pull requests.
- [ ] SARIF integration works.
- [ ] New-risk gating works.

- [ ] Errors are actionable.
- [ ] Supported examples run successfully.

Detection and prioritisation

- [ ] Findings are normalised.
- [ ] Findings are deduplicated.
- [ ] Cross-scanner evidence is correlated.
- [ ] Threat intelligence is included.
- [ ] Reachability is included where supported.
- [ ] Policy decisions are explainable.

Governance

- [ ] Finding lifecycle exists.
- [ ] Suppressions require expiry and evidence.
- [ ] Audit history is preserved.
- [ ] Policy-as-code is implemented.
- [ ] Compliance evidence clearly distinguishes automated and manual controls.

Interoperability

- [ ] SARIF is generated.
- [ ] CycloneDX SBOM is generated.
- [ ] SPDX SBOM is generated.
- [ ] VEX is generated.
- [ ] Signed evidence bundles are generated.
- [ ] JSON schemas are versioned and validated.

Remediation

- [ ] Deterministic fixes are tested.
- [ ] Guided remediation is available.
- [ ] AI remediation is optional and privacy-controlled.
- [ ] Generated fixes are verified before being marked complete.

Commercial readiness

- [ ] Product and research identities are separated.
- [ ] Editions and feature boundaries are documented.
- [ ] Licensing is reviewed.
- [ ] Data-handling modes are documented.
- [ ] Local, hybrid and self-hosted modes are defined.
- [ ] Support and incident processes are operational.

Final completion command

The implementing agent should create a final automated command similar to:

```
trustgate project verify-release
```

This command must fail unless all required release conditions pass.

It should verify:

```
schemas
unit tests
integration tests
security tests
end-to-end tests
benchmark thresholds
dependency pinning
Action pinning
SBOM generation
VEX generation
SARIF validation
documentation examples
release signatures
provenance
changelog
version consistency
```

The project must not be described as fully complete until this command exits successfully and all manual external-review requirements have been recorded.